

Anexo A — Gramáticas de los tres lenguajes

Este anexo reproduce las gramáticas ANTLR 4 que definen los tres lenguajes soportados. Los archivos fuente viven en `src/main/antlr4/` y su regeneración tiene particularidades.

Lenguaje	Extensión	Archivo(s) de gramática	Tipo de gramática
Lat (Pig Latin)	.pig	<code>src/main/antlr4/PigLatin.g4</code>	Combinada (lexer + parser)
Y?	.y	<code>src/main/antlr4/YLexer.g4</code> + <code>src/main/antlr4/YParser.g4</code>	Separada (lexer + parser)
Zetariano	.z	<code>src/main/antlr4/Z.g4</code>	Combinada (lexer + parser)

A.1 Lat — `PigLatin.g4`

```
grammar PigLatin;
```

```
// global
```

```
program
```

```
    : importDeclaration* globalVariablesSection? functionSection? mainSection  
    FINIS_PROGRAM ';' EOF
```

```
    ;
```

```
importDeclaration : IMPORT importPath; // new
```

```
importPath : ID ('.' ID)+; // new
```

```
globalVariablesSection
```

```
    : VARIABLES_SECTION declaration*
```

```
    ;
```

```
functionSection
```

: FUNCTIONS_SECTION functionDeclaration*

;

mainSection

: MAIN_SECTION statement*

;

// functions

functionDeclaration

: procedureDeclaration

| functionWithReturn

;

procedureDeclaration

: ACTIO ID '(' parameterList? ')' functionBody FINIS ';

;

functionWithReturn

: RATIO type ID '(' parameterList? ')' functionBody FINIS ';

;

parameterList

: parameters+=parameter (',' parameters+=parameter)*

;

parameter

: ESTO ID ':' type

;

functionBody

: '{' localVariableSection? statement* '}'

;

// declarations

declaration

: variableDeclaration

| arrayDeclaration

;

localVariableSection

: VARIABLES '[' declaration* ']'

;

variableDeclaration

: ESTO ID ':' type ';' #NormalVarDeclaration

| ESTO ID ':' type expression ';' #NormalVarDeclaration

| ESTO ID ':' type structInitializer ';' ? #NormalVarDeclaration

| ESTO variableName=ID ':' NOVUS objectType=ID functionArguments ';' #NewObjectDeclaration

;

initializer

: expression #ExprInit

| structInitializer #StructInit

;

structInitializer

: '{' fields+=structFieldInitializer (',' fields+=structFieldInitializer)* '}'

;

structFieldInitializer

: ID ':' initializer

;

arrayConstructor

: type '[' expression ']'

;

arrayDeclaration

: SERIES ID ('[' expression '])+ ':' type arrayInitializer? ';' ;

;

arrayInitializer

: '{' arrayInitializerElementList? '}'

;

arrayInitializerElementList

: arrayInitializerElement (',' arrayInitializerElement)*

;

arrayInitializerElement

: expression #ArrayExprElement

| arrayInitializer #NestedArrayElement

;

type

: NUMERUS

| DECIMALIS

| TEXTUM

| BOOL

| LITTERA

| ID

;

// statements

statement

: assignment #AssignmentStmt

| incrementStatement #IncrementStmt

| functionCallStatement #FunctionCallStatementStmt

| ifStatement #IfStmt

| whileStatement #WhileStmt

| doWhileStatement #DoWhileStmt

| forStatement #ForStmt

| returnStatement #ReturnStmt

| breakStatement #BreakStmt

| continueStatement #ContinueStmt

| printStatement #PrintStmt

| readStatement #ReadStmt

;

assignment

: postfixExpression '=' expression ';' ;

| postfixExpression '=' structInitializer ';' ?

;

incrementStatement

: incrementableExpression PLUSPLUS ';' ;

| incrementableExpression MINUSMINUS ';' ;

;

incrementableExpression

: ID #IncrementVariable

| postfixExpression '[' expression ']' #IncrementArrayAccess

| postfixExpression '.' ID #IncrementMemberAccess

;

functionCallStatement

: callableExpression functionArguments ';' ;

;

callableExpression

: ID #CallVariable

| postfixExpression '[' expression ']' #CallArrayAccess

| postfixExpression '.' ID #CallMemberAccess

;

ifStatement

: SI '(' expression ')' block elseifStatement* elseStatement? FINIS ';

;

elseifStatement

: ALITER '(' expression ')' block

;

elseStatement

: ALITER block

;

block

: '{' statement* '}'

;

whileStatement

: DUM '(' expression ')' block FINIS ';

;

doWhileStatement

: FACERE block DUM '(' expression ')' ';

;

forStatement

: PER '(' variableDeclaration expression ';' expression ')' block

;

returnStatement

: REDDERE expression? ';

;

continueStatement

: PERGE ';

;

breakStatement

: INTERRUPE ';

;

readStatement

: postfixExpression? READ

;

printStatement

: PRINT expressions+=expression (PRINT expressions+=expression)* ';

;

// expressions

expression

: logicalOrExpression

;

logicalOrExpression

: logicalOrExpression OR logicalAndExpression #OrExpr

| logicalAndExpression #ToLogicalAndExpr

;

logicalAndExpression

: logicalAndExpression AND equalityExpression #AndExpr

| equalityExpression #ToEqualityExpr

;

equalityExpression

: equalityExpression EQUAL comparisonExpression #EqualExpr

| equalityExpression NOT_EQUAL comparisonExpression #NotEqualExpr

| comparisonExpression #ToComparisonExpr

;

comparisonExpression

: comparisonExpression LESS additiveExpression #LessExpr

| comparisonExpression GREATER additiveExpression #GreaterExpr

| comparisonExpression LESS_EQUAL additiveExpression #LessEqualExpr

| comparisonExpression GREATER_EQUAL additiveExpression #GreaterEqualExpr

| additiveExpression #ToAdditiveExpr

;

additiveExpression

: additiveExpression PLUS multiplicativeExpression #AdditionExpr

| additiveExpression MINUS multiplicativeExpression #SubtractionExpr

| multiplicativeExpression #ToMultiplicativeExpr

;

multiplicativeExpression

```
: multiplicativeExpression MULT unaryExpression    #MultiplicationExpr  
| multiplicativeExpression DIV unaryExpression    #DivisionExpr  
| unaryExpression                                #ToUnaryExpr  
;
```

unaryExpression

```
: NON unaryExpression    #NotExpr  
| MINUS unaryExpression    #NegateExpr  
| postfixExpression      #ToPostfixExpr  
;
```

postfixExpression

```
: postfixExpression '[' expression ']'    #ArrayAccessExpr  
| postfixExpression '.' ID                #MemberAccessExpr  
| postfixExpression functionArguments    #FunctionCallExpr  
| postfixExpression PLUSPLUS              #PostIncrementExpr  
| postfixExpression MINUSMINUS            #PostDecrementExpr  
| primaryExpression                      #ToPrimaryExpr  
;
```

functionArguments

```
: '(' argumentList? ')'  
;
```

argumentList

: arguments+=expression (',' arguments+=expression)*

;

primaryExpression

: NUMBER #NumberLiteralExpr

| DECIMAL #DecimalLiteralExpr

| STRING #StringLiteralExpr

| CHAR #CharLiteralExpr

| VERUM #TrueLiteralExpr

| FALSUS #FalseLiteralExpr

| ID #VariableExpr

| NOVUS ID '(' argumentList? ')' #ObjectCreationExpr

| '(' expression ')' #ParenthesizedExpr

;

// Lexer

// key words for sections

VARIABLES_SECTION : 'VARIABLES>';

FUNCTIONS_SECTION : 'MUNERA>';

MAIN_SECTION : 'MAIOR>';

FINIS_PROGRAM : 'FINIS';

// key words

FINIS : 'finis';

ESTO : 'esto';

SERIES : 'series';

ACTIO : 'actio';

RATIO : 'ratio';

SI : 'si';

ALITER : 'aliter';

DUM : 'dum';

FACERE : 'facere';

PER : 'per';

REDDERE : 'reddere';

PERGE : 'perge';

INTERRUMPE : 'interrumpe';

VARIABLES : 'VARIABLES';

IMPORT : 'import';

NOVUS : 'novus';

// key word for types

NUMERUS : 'numerus';

DECIMALIS : 'decimalis';

TEXTUM : 'textum';

LITTERA : 'littera';

BOOL : 'bool';

VERUM : 'verum';

FALSUS : 'falsus';

// operators

PLUS : '+';

MINUS : '-';

MULT : '*';

DIV : '/';

PLUSPLUS : '++';

MINUSMINUS : '--';

AND : '&&';

OR : '||';

NON : 'non';

EQUAL : '==';

NOT_EQUAL : '!=';

LESS : '<';

GREATER : '>';

LESS_EQUAL : '<=';

GREATER_EQUAL : '>=';

READ : '<<';

PRINT : '>>';

// literals

NUMBER : [0-9]+ ;

DECIMAL : [0-9]+ '.' [0-9]+ ;

```

STRING : '"' (~["\r\n])* '"' ;

CHAR   : '\' . '\'' ;

ID      : [a-zA-Z_][a-zA-Z0-9_]* ;

// comments

LINE_COMMENT : '/' ~[\r\n]* -> channel(HIDDEN) ;

BLOCK_COMMENT : '##' .*? '##' -> channel(HIDDEN) ;

WS       : [ \t\r\n]+ -> skip ;

```

A.1.1 Observaciones de Lat

- Las secciones se delimitan con tokens especiales: VARIABILES>, MUNERA> y MAIOR>, y el programa cierra con FINIS;
- Declaración de variable: esto nombre : tipo valor; (el valor es opcional).
- Arreglos: series nombre[n] ([m]...) : tipo { ... };
- Funciones: actio nombre(params) { ... } finis; (sin retorno) y ratio tipo nombre(params) { ... } finis; (con retorno).
- Imports: import ruta.dotted.al.archivo.y|z (solo Lat importa).
- Comentarios: // de línea y ## ... ## de bloque.

A.2 Y? — YLexer.g4 + YParser.g4

A.2.1 Lexer

```

lexer grammar YLexer;

tokens { INDENT, DEDENT }

@header {

import org.antlr.v4.runtime.Token;

}

STRUCTURES : '%estructuras';

```

FUNCTIONS : '%funciones';

STRUCTURE : 'estructura';

DEFINE : 'definir';

IF : 'si';

THEN : 'entonces';

ELSE : 'sino';

OTHERWISE : 'contrario';

SWITCH : 'elegir';

CASE : 'caso';

ALWAYS : 'siempre';

FOR : 'para';

WHILE : 'mientras';

DO : 'hacer';

BREAK : 'romper';

CONTINUE : 'continuar';

RETURN : 'retornar';

PRINT : 'imprimir';

READ : 'leer';

INTEGER : 'entero';

FLOAT : 'flotante';

CHARACTER : 'caracter';

BOOLEAN : 'bool';

STRING : 'cadena';

TRUE : 'verdadero';

FALSE : 'falso';

INCREMENT : '++';

DECREMENT : '--';

PLUS : '+';

MINUS : '-';

MULT : '*';

DIV : '/';

MOD : '%';

AND : '&&';

OR : '||';

NOT : '!';

EQUAL : '==';

NOT_EQUAL : '!=';

LESS_EQUAL : '<=';

GREATER_EQUAL : '>=';

LESS : '<';

GREATER : '>';

PLUS_ASSIGN : '+=';

MINUS_ASSIGN : '-=';

```
MULT_ASSIGN : '*=';

ASSIGN      : '=';

ARROW       : '->';

LPAREN : '(';

RPAREN : ')';

LBRACK : '[';

RBRACK : ']';

LBRACE : '{';

RBRACE : '}';

COMMA : ',';

COLON : ':';

SEMI : ';';

DOT : '.';

FLOAT_LITERAL : [0-9]+ '.' [0-9]+ ;

INTEGER_LITERAL : [0-9]+ ;

CHAR_LITERAL : '\"' . '\"' ;

STRING_LITERAL : '"' (~[\"\\r\\n])* '"' ;

ID : [a-zA-Z_][a-zA-Z0-9_]* ;

LINE_COMMENT : '/' ~[\\r\\n]* -> channel(HIDDEN) ;

BLOCK_COMMENT : '/*' .*? '*' -> channel(HIDDEN) ;

NEWLINE : '\\r'? '\\n' ;

WS : [ \\t]+ ;
```

Nota. La gramática declara los tokens sintéticos INDENT/DEDENT. La producción de esos tokens la realiza la clase **escrita a mano** `lexer_parser/y/IndentationTokenSource.java`, que envuelve al lexer aplicando la regla de indentación estilo Python.

A.2.2 Parser

```
parser grammar YParser;
```

```
options {
```

```
    tokenVocab = YLexer;
```

```
}
```

```
// PROGRAMA
```

```
program : NEWLINE* structuresSection? functionsSection NEWLINE* EOF;
```

```
// SECCIONES
```

```
structuresSection : STRUCTURES NEWLINE structureDeclaration+;
```

```
functionsSection : FUNCTIONS NEWLINE functionDeclaration+;
```

```
// ESTRUCTURAS
```

```
structureDeclaration : STRUCTURE ID COLON NEWLINE INDENT structureField+ DEDENT;
```

```
structureField : type ID arrayDimensions? NEWLINE;
```

```
arrayDimensions : (LBRACK INTEGER_LITERAL RBRACK)+;
```

```
// FUNCIONES
```

```
functionDeclaration : DEFINE ID LPAREN parameterList? RPAREN returnType? COLON  
NEWLINE INDENT statement* DEDENT;
```

```
returnType : ARROW type;
```

```
parameterList : parameter (COMMA parameter)*;
```

parameter

: arrayParameter	#ArrayParameterParam
structureParameter	#StructureParameterParam
type ID	#NormalParameter
;	

arrayParameter : LBRACK RBRACK type ID;

structureParameter : LBRACE RBRACE ID ID;

// SENTENCIAS

statement

: variableDeclaration	#VariableDeclarationStmt
structureDeclaration	#StructureDeclarationStmt
assignmentStatement	#AssignmentStmt
incrementStatement	#IncrementStmt
ifStatement	#IfStmt
switchStatement	#SwitchStmt
forStatement	#ForStmt
whileStatement	#WhileStmt
doWhileStatement	#DoWhileStmt
breakStatement	#BreakStmt
continueStatement	#ContinueStmt
returnStatement	#ReturnStmt
printStatement	#PrintStmt

| readStatement #ReadStmt

| expressionStatement #ExpressionStmt

;

// DECLARACIONES

variableDeclaration

: variableDeclarationCore NEWLINE

;

variableDeclarationCore

: type ID initializer?

| type ID arrayDeclaration initializer?

;

arrayDeclaration : (LBRAK expression RBRAK)+;

initializer

: ASSIGN expression #ExpressionInitializer

| ASSIGN arrayInitializer #ArrayInitializerValue

| ASSIGN structureInitializer #StructureInitializerValue

;

arrayInitializer : LBRAK arrayInitializerElementList? RBRAK;

arrayInitializerElementList

: arrayInitializerElement (COMMA arrayInitializerElement)*

;

arrayInitializerElement

: expression #ArrayExprElement
| arrayInitializer #NestedArrayElement
;

structureInitializer : LBACE expressionList? RBACE;

expressionList : expression (COMMA expression)*;

// ASIGNACIONES

assignmentStatement : assignableExpression assignmentOperator expression NEWLINE;

assignmentOperator

: ASSIGN #AssignOperator
| PLUS_ASSIGN #PlusAssignOperator
| MINUS_ASSIGN #MinusAssignOperator
| MULT_ASSIGN #MultiplyAssignOperator
;

// INCREMENTO / DECREMENTO

incrementStatement : assignableExpression (INCREMENT | DECREMENT) NEWLINE;

assignableExpression : postfixExpression;

// IF

ifStatement : IF LPAREN expression RPAREN THEN NEWLINE INDENT statement* DEDENT
elseifClause* elseClause?;

elseifClause : ELSE LPAREN expression RPAREN THEN NEWLINE INDENT statement*
DEDENT;

elseClause : OTHERWISE NEWLINE INDENT statement* DEDENT;

// SWITCH

switchStatement : SWITCH LPAREN expression RPAREN COLON NEWLINE INDENT
switchCase* defaultCase? DEDENT;

switchCase : CASE expression COLON NEWLINE INDENT statement* breakStatement
DEDENT;

defaultCase : ALWAYS COLON NEWLINE INDENT statement* breakStatement DEDENT;

// FOR

forStatement : FOR LPAREN forInitializer? SEMI expression? SEMI forUpdate? RPAREN
COLON NEWLINE INDENT statement* DEDENT;

forInitializer

: variableDeclarationCore

| expression

;

variableDeclarationWithoutNewline

: type ID initializer?

| type ID arrayDeclaration initializer?

| ID ID initializer?

;

forUpdate : expression;

// WHILE

whileStatement : WHILE LPAREN expression RPAREN DO COLON NEWLINE INDENT
statement* DEDENT;

// DO-WHILE

doWhileStatement : DO COLON NEWLINE INDENT statement* DEDENT WHILE LPAREN
expression RPAREN NEWLINE;

// CONTROL DE FLUJO

breakStatement : BREAK NEWLINE;

continueStatement : CONTINUE NEWLINE;

returnStatement : RETURN expression? NEWLINE;

// FUNCIONES ESPECIALES

printStatement : PRINT LPAREN expressionList? RPAREN NEWLINE;

readStatement : READ LPAREN RPAREN NEWLINE;

expressionStatement : expression NEWLINE;

// EXPRESIONES

expression

: logicalOrExpression

;

logicalOrExpression

: logicalOrExpression OR logicalAndExpression #OrExpr

| logicalAndExpression #ToLogicalAndExpr

;

logicalAndExpression

: logicalAndExpression AND equalityExpression #AndExpr

| equalityExpression #ToEqualityExpr

;

equalityExpression

```
: equalityExpression EQUAL comparisonExpression    #EqualExpr  
| equalityExpression NOT_EQUAL comparisonExpression #NotEqualExpr  
| comparisonExpression                            #ToComparisonExpr  
;
```

comparisonExpression

```
: comparisonExpression LESS additiveExpression      #LessExpr  
| comparisonExpression GREATER additiveExpression    #GreaterExpr  
| comparisonExpression LESS_EQUAL additiveExpression #LessEqualExpr  
| comparisonExpression GREATER_EQUAL additiveExpression #GreaterEqualExpr  
| additiveExpression                                #ToAdditiveExpr  
;
```

additiveExpression

```
: additiveExpression PLUS multiplicativeExpression #AdditionExpr  
| additiveExpression MINUS multiplicativeExpression #SubtractionExpr  
| multiplicativeExpression                          #ToMultiplicativeExpr  
;
```

multiplicativeExpression

```
: multiplicativeExpression MULT unaryExpression    #MultiplicationExpr  
| multiplicativeExpression DIV unaryExpression      #DivisionExpr  
| multiplicativeExpression MOD unaryExpression      #ModuloExpr  
| unaryExpression                                  #ToUnaryExpr
```

;

unaryExpression

: NOT unaryExpression #NotExpr
| MINUS unaryExpression #NegateExpr
| postfixExpression #ToPostfixExpr

;

postfixExpression

: primaryExpression postfixOperation*

;

postfixOperation

: LBRACK expression RBRACK #ArrayAccessOp
| DOT ID #MemberAccessOp
| LPAREN argumentList? RPAREN #FunctionCallOp
| INCREMENT #PostIncrementOp
| DECREMENT #PostDecrementOp

;

primaryExpression

: INTEGER_LITERAL #IntegerLiteralExpr
| FLOAT_LITERAL #FloatLiteralExpr
| CHAR_LITERAL #CharLiteralExpr
| STRING_LITERAL #StringLiteralExpr

```

| TRUE                                #TrueLiteralExpr

| FALSE                              #FalseLiteralExpr

| ID                                  #VariableExpr

| READ LPAREN RPAREN                 #ReadExpr

| LPAREN expression RPAREN           #ParenthesizedExpr

;

```

argumentList

```

: expression (COMMA expression)*

;

```

// TIPOS

type

```

: INTEGER                            #IntegerType

| FLOAT                              #FloatType

| CHARACTER                          #CharacterType

| BOOLEAN                            #BooleanType

| STRING                             #StringType

| ID                                 #StructureType

;

```

A.2.3 Observaciones de Y

- El lenguaje es **sensible a mayúsculas** y las **tabulaciones** definen el orden del código (INDENT/DEDENT).
- Se compone de dos secciones opcionales/obligatorias: %estructuras (opcional) y %funciones (obligatoria).
- No se permiten variables globales; todo vive dentro de funciones.

- Los arreglos se reciben como [] tipo nombre y las estructuras como { } Estructura nombre; ambos se pasan **por referencia**.
- Comentarios // y /* ... */.

A.3 Zetariano — Z.g4

grammar Z;

program : classDeclaration EOF;

classDeclaration : PUBLIC CLASS ID LBRACE classMember* RBRACE;

classMember

: fieldDeclaration #FieldMember

| constructorDeclaration #ConstructorMember

| methodDeclaration #MethodMember

;

fieldDeclaration : PUBLIC? type variableDeclaratorList SEMI;

variableDeclaratorList : variableDeclarator (COMMA variableDeclarator)*;

variableDeclarator : ID initializer?;

constructorDeclaration : PUBLIC ID LPAREN parameterList? RPAREN block;

methodDeclaration

: PUBLIC? type ID LPAREN parameterList? RPAREN block #TypedMethodDeclaration

| PUBLIC? VOID ID LPAREN parameterList? RPAREN block #VoidMethodDeclaration

;

parameterList : parameter (COMMA parameter)*;

parameter : type ID;

block : LBRACE statement* RBACE;

statement

: variableDeclarationStatement

| expressionStatement

| printStatement

| printlnStatement

| readlnStatement

| ifStatement

| switchStatement

| forStatement

| whileStatement

| doWhileStatement

| returnStatement

| breakStatement

| continueStatement

;

variableDeclarationStatement : type variableDeclaratorList SEMI;

initializer

: ASSIGN expression #ExpressionInitializerValue

| ASSIGN arrayInitializer #ArrayInitializerValue

;

statementOrBlock

: statement

| block

;

ifStatement : IF LPAREN expression RPAREN statementOrBlock (ELSE IF LPAREN expression RPAREN statementOrBlock)* (ELSE statementOrBlock)?;

switchStatement : SWITCH LPAREN expression RPAREN LBRACE switchCase* defaultCase? RBRACE;

switchCase : CASE expression COLON statement*;

defaultCase : DEFAULT COLON statement*;

forStatement : FOR LPAREN forInitializer? SEMI expression? SEMI forUpdate? RPAREN block;

forInitializer

: variableDeclarationStatementNoSemi

| expressionList

;

variableDeclarationStatementNoSemi : type variableDeclaratorList;

forUpdate : expressionList;

expressionList : expression (COMMA expression)*;

whileStatement : WHILE LPAREN expression RPAREN block;

doWhileStatement : DO block WHILE LPAREN expression RPAREN SEMI;

returnStatement : RETURN expression? SEMI;

breakStatement : BREAK SEMI;

continueStatement : CONTINUE SEMI;

printStatement : PRINT LPAREN argumentList? RPAREN SEMI;

printlnStatement : PRINTLN LPAREN argumentList? RPAREN SEMI;

readlnStatement : READLN LPAREN RPAREN SEMI;

expressionStatement : expression SEMI;

expression : assignmentExpression;

assignmentExpression

: conditionalExpression #ToConditionalExpression

| postfixExpression assignmentOperator assignmentExpression
#AssignmentExpressionValue

;

assignmentOperator

: ASSIGN

| PLUS_ASSIGN

| MINUS_ASSIGN

| MULT_ASSIGN

;

conditionalExpression

: logicalOrExpression #LogicalOrConditionalExpression

| logicalOrExpression QUESTION expression COLON expression #TernaryExpression

;

logicalOrExpression

: logicalOrExpression OR logicalAndExpression #OrExpr

| logicalAndExpression #ToLogicalAndExpr

;

logicalAndExpression

: logicalAndExpression AND equalityExpression #AndExpr

| equalityExpression #ToEqualityExpr

;

equalityExpression

: equalityExpression EQUAL comparisonExpression #EqualExpr

| equalityExpression NOT_EQUAL comparisonExpression #NotEqualExpr

| comparisonExpression #ToComparisonExpr

;

comparisonExpression

: comparisonExpression LESS additiveExpression #LessExpr

| comparisonExpression GREATER additiveExpression #GreaterExpr

| comparisonExpression LESS_EQUAL additiveExpression #LessEqualExpr

| comparisonExpression GREATER_EQUAL additiveExpression #GreaterEqualExpr

| additiveExpression #ToAdditiveExpr

;

additiveExpression

: additiveExpression PLUS multiplicativeExpression #AdditionExpr

| additiveExpression MINUS multiplicativeExpression #SubtractionExpr

| multiplicativeExpression #ToMultiplicativeExpr

;

multiplicativeExpression

: multiplicativeExpression MULT unaryExpression #MultiplicationExpr
| multiplicativeExpression DIV unaryExpression #DivisionExpr
| multiplicativeExpression MOD unaryExpression #ModuloExpr
| unaryExpression #ToUnaryExpr

;

unaryExpression

: NOT unaryExpression #NotExpr
| MINUS unaryExpression #NegateExpr
| PLUS unaryExpression #PositiveExpr
| postfixExpression #ToPostfixExpr

;

postfixExpression

: primaryExpression postfixOperation*

;

postfixOperation

: LBRACK expression RBRACK #ArrayAccessOp
| DOT ID #MemberAccessOp
| LPAREN argumentList? RPAREN #FunctionCallOp
| INCREMENT #PostIncrementOp

| DECREMENT #PostDecrementOp

;

primaryExpression

: literal #LiteralExpr

| ID #VariableExpr

| NEW ID LPAREN argumentList? RPAREN #ObjCreationExpr

| arrayCreation #ArrayCreationExpr

| NULL #NullExpr

| READLN LPAREN RPAREN #ReadExpr

| LPAREN expression RPAREN #ParenthesizedExpr

;

arrayCreation

: NEW primitiveType arrayCreationDimensions

| NEW ID arrayCreationDimensions

;

arrayCreationDimensions

: (LBRACK expression RBRACK)+

;

literal

: INTEGER_LITERAL #IntegerLiteralExpr

| FLOAT_LITERAL #FloatLiteralExpr

| CHAR_LITERAL #CharLiteralExpr

| STRING_LITERAL #StringLiteralExpr

| TRUE #TrueLiteralExpr

| FALSE #FalseLiteralExpr

;

argumentList : expression (COMMA expression)*;

arrayInitializer : LBRACE arrayInitializerElementList? RBRACE;

arrayInitializerElementList

 : arrayInitializerElement (COMMA arrayInitializerElement)*

;

arrayInitializerElement

 : expression

 | arrayInitializer

;

arrayDimensions : (LBRACK RBRACK)+;

type

 : primitiveType arrayDimensions?

 | referenceType arrayDimensions?

;

primitiveType

 : INT

 | DOUBLE

| CHAR

| BOOLEAN

;

referenceType

: STRING

| ID

;

PUBLIC : 'public';

CLASS : 'class';

VOID : 'void';

RETURN : 'return';

IF : 'if';

ELSE : 'else';

SWITCH : 'switch';

CASE : 'case';

DEFAULT : 'default';

FOR : 'for';

WHILE : 'while';

DO : 'do';

BREAK : 'break';

CONTINUE : 'continue';

PRINT : 'print';

PRINTLN : 'println';

READLN : 'readln';

NEW : 'new';

NULL : 'null';

TRUE : 'true';

FALSE : 'false';

INT : 'int';

DOUBLE : 'double';

CHAR : 'char';

BOOLEAN : 'boolean';

STRING : 'String';

PLUS : '+';

MINUS : '-';

MULT : '*';

DIV : '/';

MOD : '%';

INCREMENT : '++';

DECREMENT : '--';

AND : '&';

OR : '||';

NOT : '!';

EQUAL : '==';

NOT_EQUAL : '!=';

LESS : '<';

GREATER : '>';

LESS_EQUAL : '<=';

GREATER_EQUAL : '>=';

ASSIGN : '=';

PLUS_ASSIGN : '+=';

MINUS_ASSIGN : '-=';

MULT_ASSIGN : '*=';

QUESTION : '?';

LPAREN : '(';

RPAREN : ')';

LBRACE : '{';

RBRACE : '}';

LBRACK : '[';

RBRACK : ']';

DOT : '.';

COMMA : ',';

COLON : ':';

SEMI : ';';

FLOAT_LITERAL

: [0-9]+ '.' [0-9]+

;

INTEGER_LITERAL

: [0-9]+

;

CHAR_LITERAL

: \" (~[\\] | \" .) \"

;

STRING_LITERAL

: \" (~[\"\\\\r\\n] | \" .)* \"

;

ID

: [a-zA-Z_][a-zA-Z0-9_]*

;

LINE_COMMENT

: '/' ~[\\r\\n]* -> channel(HIDDEN)

;

BLOCK_COMMENT

: '/*'.*? '*/' -> channel(HIDDEN)

;

WS

: [\\t\\r\\n]+ -> skip

;

A.3.1 Observaciones de Z

- Un archivo .z contiene **exactamente una clase**; el nombre de la clase debe coincidir con el nombre del archivo.
- Solo se contemplan atributos (public opcional), constructor(es) y métodos; **no** hay herencia, encapsulamiento ni polimorfismo en esta entrega.
- Sobrecarga de constructores y métodos soportada (se resuelve por firma).
- new crea objetos en el **heap**; new tipo[n] crea arreglos.
- Funciones del sistema: print, println, readln.

A.4 Convenciones de precedencia (comunes)

De menor a mayor precedencia en las expresiones de los tres lenguajes:

1. ||
2. &&
3. == !=
4. < > <= >=
5. + -
6. * / % (Lat no tiene %)
7. Unarios ! - (Z además +)
8. Postfijos: [], ., llamada (), ++, --
9. Primarios y paréntesis

En Z, además, la expresión ternaria cond ? val1 : val2 y la asignación de expresión tienen la precedencia más baja (asignación por debajo del ternario).